

Navigatore Burocratico — Analisi Tecnica

Versione documento: 0.3

Data: 2026-03-27

Stato: Draft

1. Vision & Scope

Navigatore Burocratico è un'applicazione mobile-first che guida cittadini e professionisti italiani attraverso le procedure burocratiche passo per passo.

MVP: Regione Calabria — Provincia di Cosenza.

L'obiettivo è abbattere la complessità della burocrazia italiana fornendo:

- Percorsi guidati per ogni pratica (es. DIA, SCIA, permesso di costruire, successioni, ecc.)
- Lista documenti necessari per ogni step
- Indicazioni su uffici competenti, costi, tempi stimati
- Assistenza intelligente tramite LLM locale (Ollama)

2. Stack Tecnologico

2.1 Backend

Componente	Tecnologia	Versione
Linguaggio	Python	3.13
Framework API	FastAPI	≥ 0.115
ORM	SQLAlchemy	≥ 2.0
Migrazioni DB	Alembic	≥ 1.14
Server ASGI	Uvicorn	≥ 0.30
Configurazione	Pydantic Settings	≥ 2.7
Upload file	python-multipart	≥ 0.0.20
HTTP client	HTTPX	≥ 0.28
AI / LLM	Ollama (self-hosted)	—

2.2 Mobile

Componente	Tecnologia
Framework	Flutter

Componente	Tecnologia
Target	iOS + Android

2.3 Infrastruttura (pianificata)

Componente	Tecnologia
Database	PostgreSQL (prod + dev via Docker)
Container	Docker + Docker Compose
CI/CD	GitHub Actions
Deploy backend	TBD (VPS / Railway / Render)

3. Architettura del Backend

3.1 Struttura attuale (scaffold)

```
backend/
  api/
    v1/          # Router FastAPI – da popolare
  core/
    config.py    # Settings via pydantic-settings
    db/          # Session SQLAlchemy + setup Alembic
    models/      # Modelli ORM
    services/    # Business logic
    main.py      # Entry point FastAPI
```

3.2 Struttura dati procedure

```
data/
  procedures/
    calabria/
      cosenza/  # File JSON/YAML delle procedure
```

3.3 Flusso architetturale

```
App Flutter
  ▼ HTTPS (REST JSON)
FastAPI Backend
  api/v1/      → Route handlers (thin layer)
  services/    → Business logic
  models/      → ORM / schema Pydantic
  db/          → PostgreSQL
  Procedure definitions (JSON/YAML su disco o in DB)
  Ollama (HTTP interno) → LLM locale per assistenza
```

4. Variabili d'Ambiente

Variabile	Descrizione
APP_ENV	development / production
SECRET_KEY	Chiave per JWT / sessioni
DATABASE_URL	URI connessione al DB
OLLAMA_BASE_URL	URL base istanza Ollama (es. http://localhost:11434)
OLLAMA_MODEL	Nome modello Ollama (es. llama3, mistral)
POSTGRES_USER	Utente PostgreSQL (Docker Compose)
POSTGRES_PASSWORD	Password PostgreSQL (Docker Compose)

5. Roadmap

Fase 0 — Fondamenta (completata)

- Scaffold backend (FastAPI, SQLAlchemy, Alembic, config)
- Struttura cartelle procedure (calabria/cosenza)
- Setup Docker Compose (backend + PostgreSQL)
- CI GitHub Actions (lint + test)
- Prima migrazione Alembic (placeholder vuoto a1b2c3d4e5f6 creato — schema reale definito in Fase 1)

Fase 1 — Procedure Core (MVP)

- Definire schema JSON/YAML per una procedura (step, documenti, ufficio, tempi, costi)
- Inserire prime procedure Cosenza (es. SCIA Edilizia, Permesso di Costruire)
- Modello ORM: Procedure, Step, Document
- API REST: GET /v1/procedures, GET /v1/procedures/{id}, GET /v1/procedures/{id}/steps
- Seed DB con dati iniziali

Fase 2 — Assistente AI

- Integrazione Ollama: servizio AIService con client HTTPX
- Endpoint POST /v1/chat (context-aware sulla procedura attiva)
- Prompt engineering per risposte focalizzate su pratica burocratica
- Rate limiting / gestione errori Ollama

Fase 3 — Autenticazione & Utenti

- Modello User + JWT auth (FastAPI OAuth2)
- Endpoint registrazione / login / refresh token
- Salvataggio progresso utente per pratica (procedura in corso, step completati)

Fase 4 — App Flutter (MVP)

- Setup progetto Flutter
- Schermata home: lista categorie procedure
- Schermata procedura: step-by-step con checklist documenti
- Schermata chat con assistente AI
- Gestione stato offline (cache locale procedure)

Fase 5 — Qualità & Deploy

- Test di integrazione API (pytest + httpx AsyncClient)
- Dockerizzazione completa (multi-stage build)
- Deploy backend su server (VPS o PaaS)
- Deploy app Flutter su store (TestFlight / Play Console beta)

Backlog / Future estensioni

- Supporto altre province calabresi
 - Supporto altre regioni italiane
 - Notifiche push (scadenze pratiche, aggiornamenti normative)
 - Upload e gestione documenti utente (storage S3-compatible)
 - Integrazione con servizi PA (SUAP, sportello unico, ecc.)
 - Versione web (PWA o React/Next.js)
-

6. Funzionalità Pianificate — Dettaglio

6.1 Catalogo Procedure

Ogni procedura è descritta da:

- **Nome e categoria** (edilizia, successioni, licenze commerciali, ecc.)
- **Ente competente** (Comune, Provincia, Regione, SUAP, ecc.)
- **Step sequenziali** numerati, ognuno con:
 - Descrizione azione
 - Documenti necessari (nome, obbligatorio/opzionale, note)
 - Ufficio / sportello di riferimento
 - Costo stimato
 - Tempo stimato
- **Tag** per filtraggio (es. **edilizia**, **privato**, **impresa**)

6.2 Assistente AI (Ollama)

- Risponde a domande contestuali sulla procedura attiva
- Accesso solo a knowledge base locale (nessuna chiamata esterna)
- Gestione "non so" con rimando a fonti ufficiali
- Possibile RAG su PDF normativi locali

6.3 Progresso Utente

- Salvataggio step completati per pratica
 - Checklist documenti con spunta
 - Riepilogo stato ("hai completato 3/7 step")
-

7. Bug Noti / Rischi Tecnici

ID	Tipo	Descrizione	Priorità
RISK-01	Dati	Le procedure burocratiche variano per comune e cambiano frequentemente — necessario processo di aggiornamento dati	Alta
RISK-02	AI	Ollama richiede hardware locale adeguato in produzione (GPU o CPU potente) — valutare alternativa cloud LLM	Alta
RISK-03	Conformità	Informazioni burocratiche errate possono causare problemi reali all'utente — disclaimer + revisione umana	Alta
RISK-04	Offline	App Flutter deve funzionare parzialmente offline — strategia cache da definire	Media
RISK-05	Auth	JWT secret key management in produzione — usare secrets manager	Media
RISK-06	CI	Il job <code>test</code> in CI imposta <code>DATABASE_URL</code> ma non avvia un container Postgres — i test DB falliranno in CI quando aggiunti	Media
RISK-07	Deploy	Il <code>Dockerfile</code> avvia direttamente uvicorn senza eseguire <code>alembic upgrade head</code> — lo schema DB non viene creato automaticamente all'avvio del container	Media

8. Convenzioni di Sviluppo

- **Branch:** `main` (stabile) → PR da branch `fase-N`; `docs` per documentazione
- **Commit:** Conventional Commits (`feat:`, `fix:`, `chore:`, `docs:`)
- **Linting:** ruff (configurato in `pyproject.toml`, regole E/F/I, line-length 100); dart format per Flutter (TBD)
- **Test:** pytest + pytest-asyncio per backend; flutter test per mobile
- **Lingua codice:** inglese (nomi variabili, commenti, commit); italiano per documentazione utente e dati procedure